facebook

Kernel Transport Layer Security

A TLS socket



Dave Watson

davejwatson@fb.com

TLS implemented as a socket

```
int tls_fd = socket(AF_TLS,
                    SOCK_STREAM | SOCK_DGRAM,
                    0);
```

Why TLS?

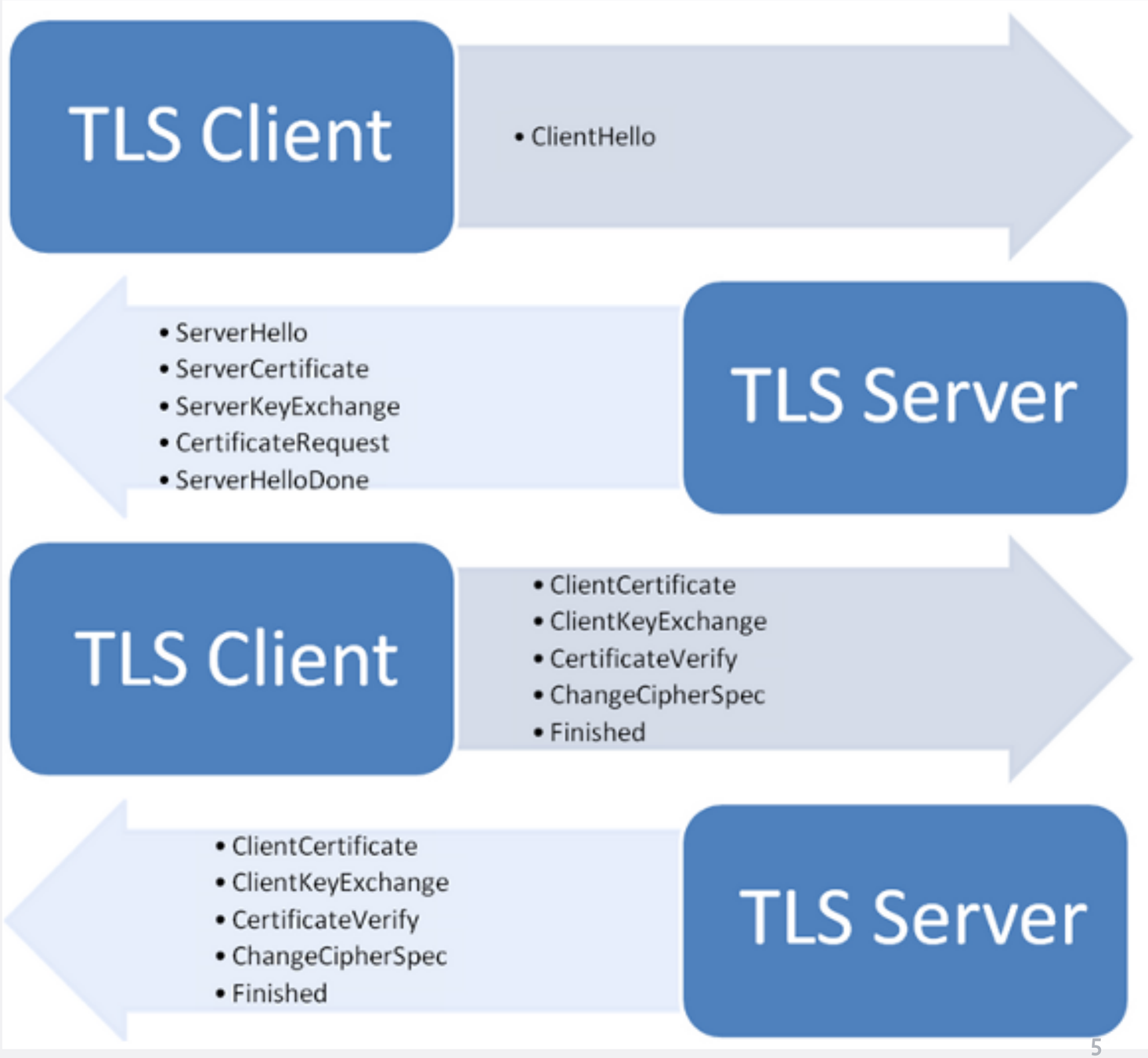
Security for the web

- The S in HTTPS
- HTTP/2 standard
- Widely deployed

- Versions: SSL 1, 2, 3, TLS 1.1, 1.2, draft 1.3

What?

What is TLS?


TLS Client
• ClientHello
TLS Server
• ServerHello
• ServerCertificate
• ServerKeyExchange
• CertificateRequest
• ServerHelloDone
TLS Client
• ClientCertificate
• ClientKeyExchange
• CertificateVerify
• ChangeCipherSpec
• Finished
TLS Server
• ClientCertificate
• ClientKeyExchange
• CertificateVerify
• ChangeCipherSpec
• Finished

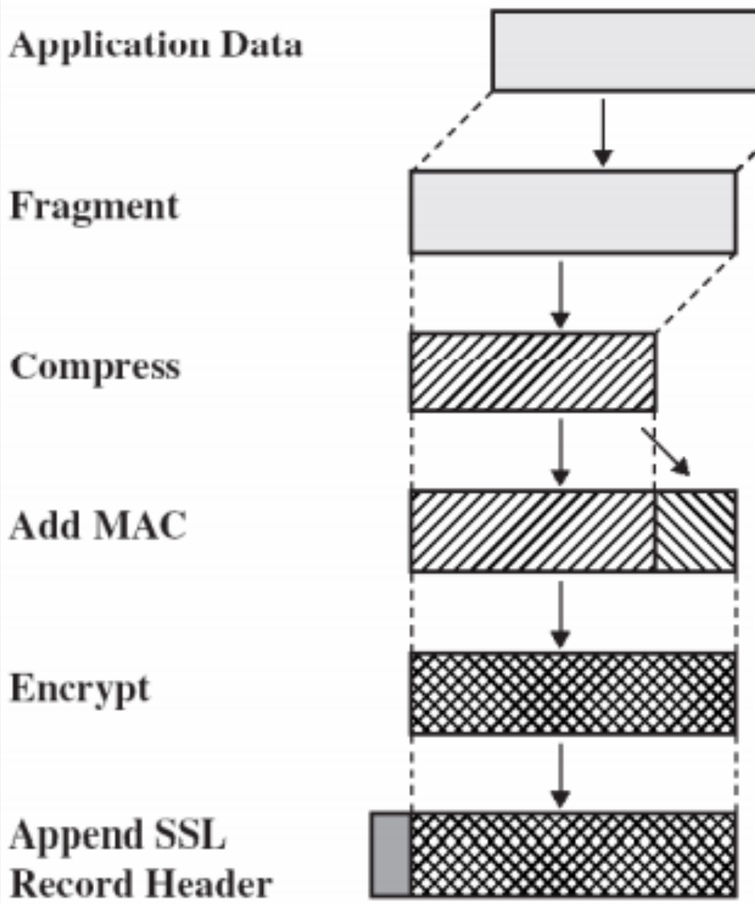
Symmetric crypto only

Last 7 OpenSSL Vulnerabilities

- Leave handshake in userspace
  - 1,2,3 - last major issues in handshake
- Leave control messages in userspace
  - 4, 5, 6 - last major bugs in control message issues
  - heartbleed, etc
- 7 – Cipher suites still controlled in userspace

What is TLS?

Record Protocol operation



Record Protocol:
- takes messages to be transmitted from apps
- fragments the data into manageable blocks
- optionally compresses the data
- applies a MAC
- encrypts
- and transmits the result.

- Reverse operation at reception (decryption, verification, decompression, reassembly, delivery to apps)

What is TLS?

Framing

- Max frame size of 16384 bytes
- 13 byte header – unaligned, needs padding
- 16 byte MAC
- 1st byte – message type
- Next 4 bytes – size
- Next 8 bytes – nonce (or sequence number)

What is TLS?

Control plane

- Data messages and control messages
  - Renegotiate
  - Shutdown
  - Alerts
  - Heartbeat (you have have heard of heartbleed)

What is TLS?

Control plane

- Data messages and control messages
  - Use same channel synchronously
  - Use same sequence numbers

- In practice, almost all control messages can be ignored
- Just close the connection

What is DTLS?

Datagrams

- Sliding window, used only for replay protection
- Sequence numbers become explicit

- Otherwise, roughly the same as TLS.

QUIC

TLS 1.3

- “[Quic’s security] will be replaced by TLS 1.3 in the future”
- Adds zero round-trip handshake

- FB has its own zero-rt crypto tests, initial results promising
  (basically Quic over TCP, also moving to TLS 1.3)

Cipher suites

TLS 1.3

- TLS 1.3 proposal has only two
  - GCM(AES)
  - Chacha20 / Poly
- Both already supported in crypto subsystem

Cipher suites

TLS 1.3

- TLS 1.3 proposal has only two
  - GCM(AES) – 80%
  - Chacha20 / Poly 6% (mobile)

- Non-TLS 1.3 Ciphers - ~.1%

Why?

Why kernel TLS?

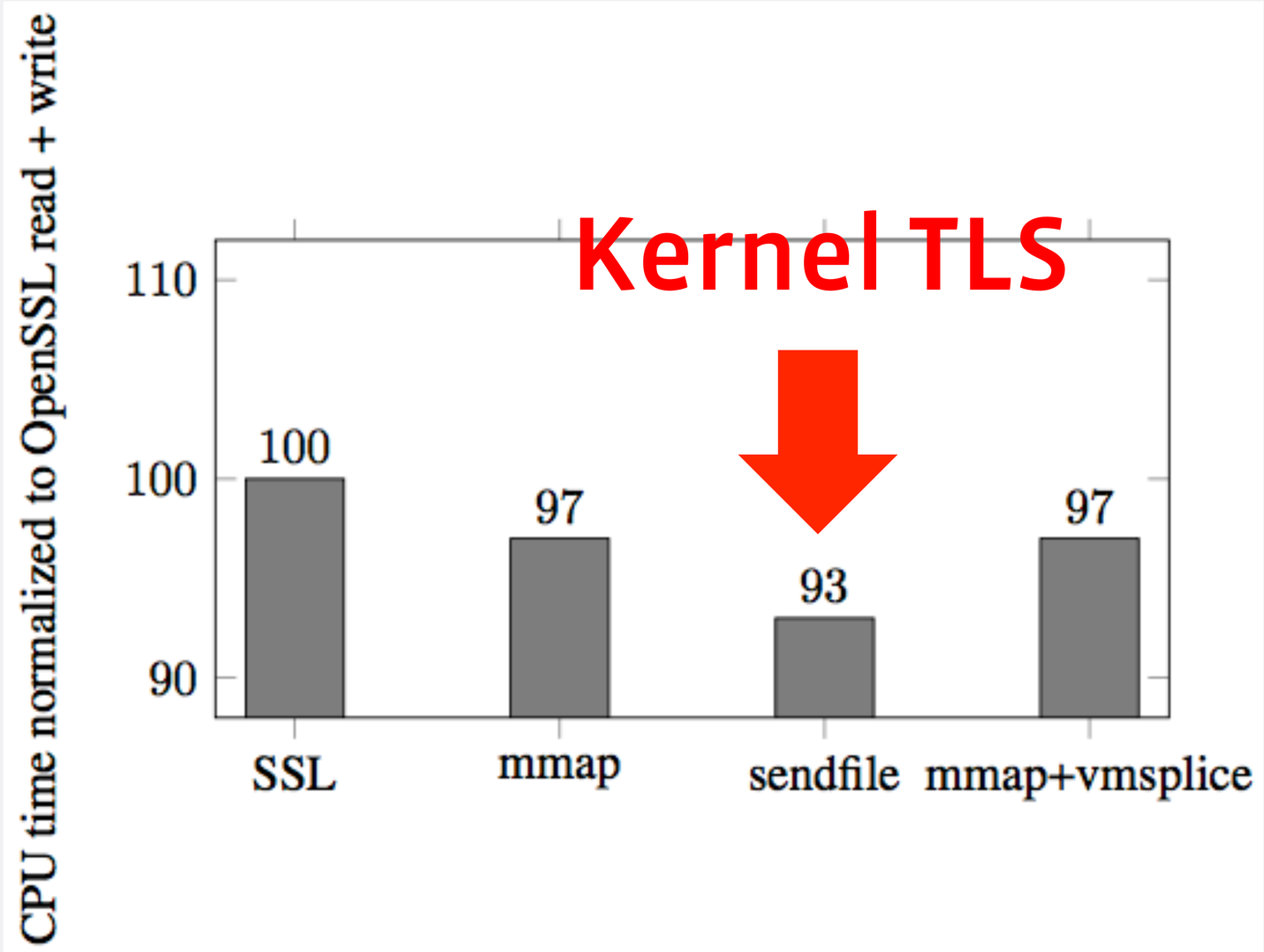
- **Sendfile, splice, standard posix APIs**

KTLS sendfile

Microbench

% CPU CYCLES

NORMALIZED TO Open SSL



KTLS sendfile

Microbench

sendfile

+ 72.38% client [aesni_intel] [k] _encrypt_by_8_new8 a
+ 2.49% client [aesni_intel] [k] _encrypt_by_88 a
+ 1.45% client [aesni_intel] [k] _get_AAD_loop2_done1962 a
+ 0.74% client [kernel.vmlinux] [k] memcpy_erms a
+ 0.72% client [kernel.vmlinux] [k] sg_nents_for_len a
+ 0.71% client [ktls] [k] tls_sendpage

mmap / SSL_write

+ 46.60% client libcrypto.so.1.0.1e [.] gcm_ghash_clmul
+ 25.82% client libcrypto.so.1.0.1e [.] aesni_ctr32_encrypt_blocks
+ 5.44% client [kernel.vmlinux] [k] copy_user_enhanced_fast_string
+ 0.94% client [kernel.vmlinux] [k] tcp_sendmsg
+ 0.76% client [kernel.vmlinux] [k] filemap_map_pages

KTLS sendfile

Microbench

sendfile

+ 72.38% client [aesni_intel] [k] _encrypt_by_8_new8 a
+ 2.49% client [aesni_intel] [k] _encrypt_by_88 a
+ 1.45% client [aesni_intel] [k] _get_AAD_loop2_done1962 a
+ 0.74% client [kernel.vmlinux] [k] memcpy_erms a
+ 0.72% client [kernel.vmlinux] [k] sg_nents_for_len a
+ 0.71% client [ktls] [k] tls_sendpage

mmap / SSL_write

+ 46.60% client libcrypto.so.1.0.1e [.] gcm_ghash_clmul
+ 25.82% client libcrypto.so.1.0.1e [.] aesni_ctr32_encrypt_blocks
+ 5.44% client [kernel.vmlinux] [k] copy_user_enhanced_fast_string
+ 0.94% client [kernel.vmlinux] [k] tcp_sendmsg
+ 0.76% client [kernel.vmlinux] [k] filemap_map_pages

- Roughly the same aesni encrypt cost

KTLS sendfile

Microbench

sendfile

+ 72.38% client [aesni_intel] [k] _encrypt_by_8_new8 a
+ 2.49% client [aesni_intel] [k] _encrypt_by_88 a
+ 1.45% client [aesni_intel] [k] _get_AAD_loop2_done1962 a
+ 0.74% client [kernel.vmlinux] [k] memcpy_erms a
+ 0.72% client [kernel.vmlinux] [k] sg_nents_for_len a
+ 0.71% client [ktls] [k] tls_sendpage

mmap / SSL_write

+ 46.60% client libcrypto.so.1.0.1e [.] gcm_ghash_clmul
+ 25.82% client libcrypto.so.1.0.1e [.] aesni_ctr32_encrypt_blocks
+ 5.44% client [kernel.vmlinux] [k] copy_user_enhanced_fast_string
+ 0.94% client [kernel.vmlinux] [k] tcp_sendmsg
+ 0.76% client [kernel.vmlinux] [k] filemap_map_pages

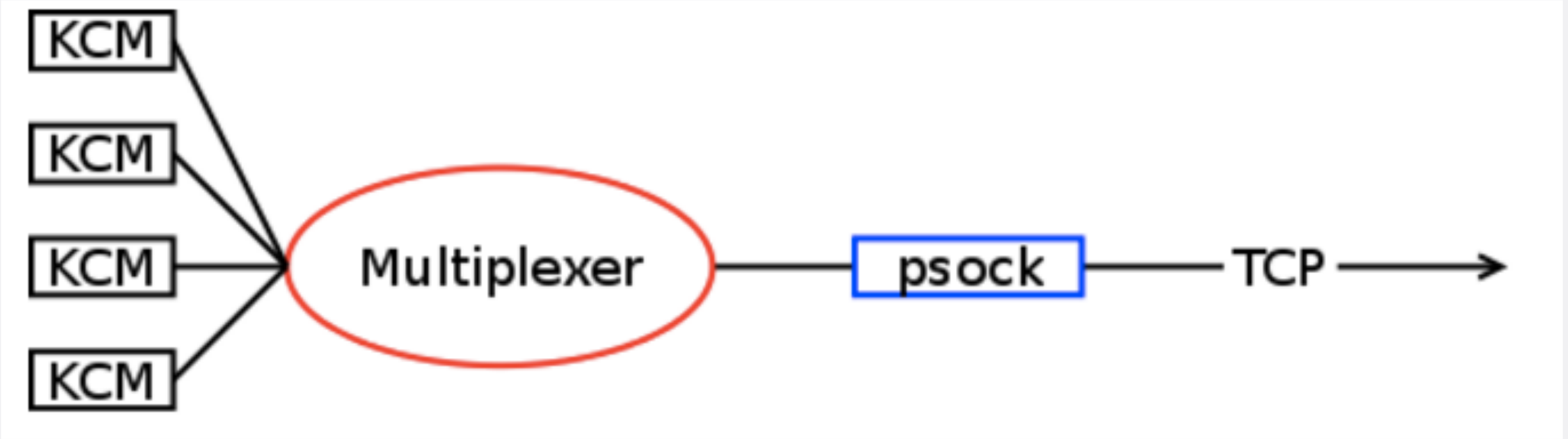
- Roughly the same aesni encrypt cost
- extra copy_user, map_pages

Why kernel TLS?

- Sendfile, splice, standard posix APIs
- **Access to unencrypted bytes in kernel – e.g., BPF**

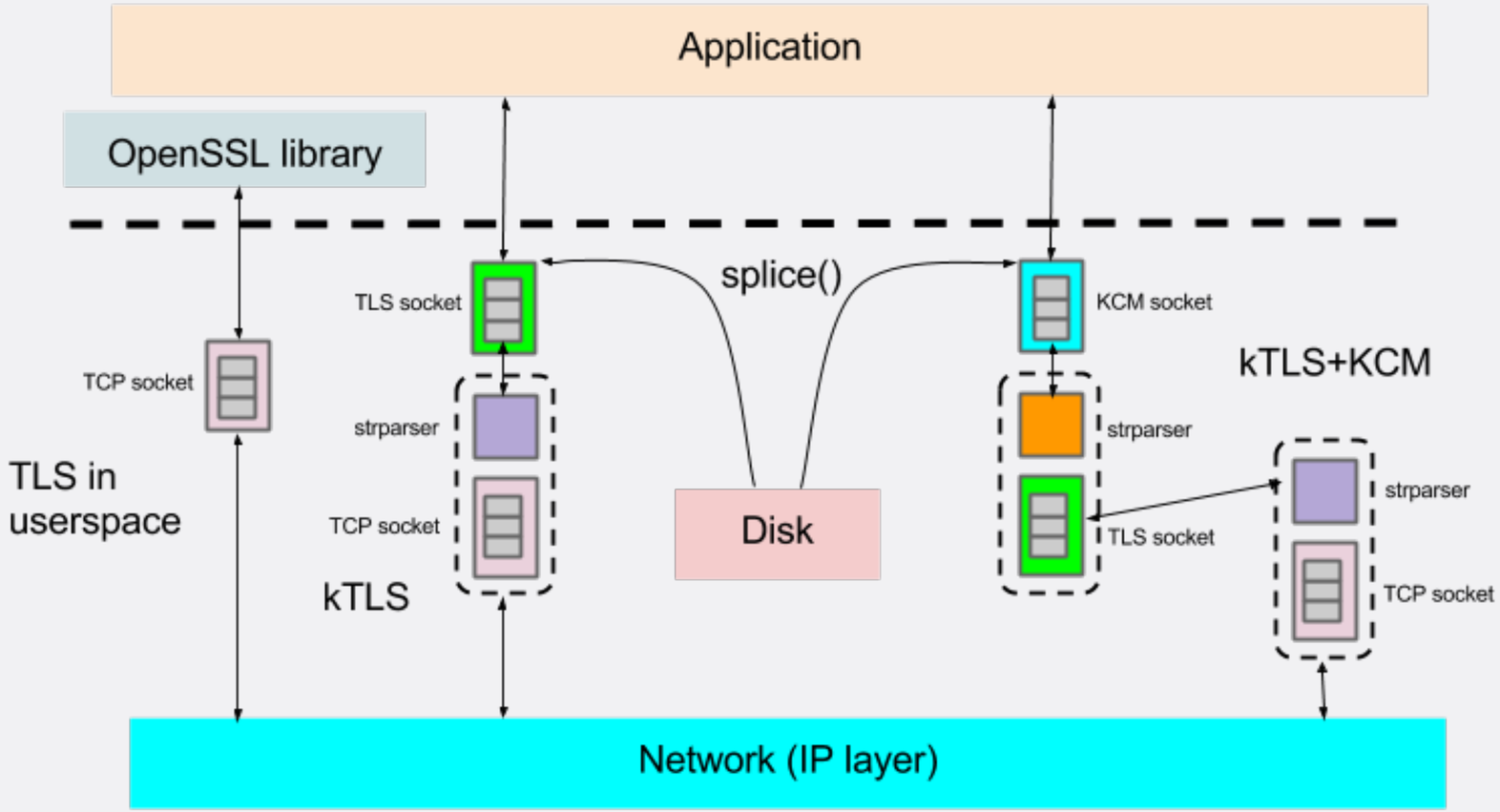
KCM

Tail latency reduction w/ Kernel Connection Multiplexor



KCM

Tail latency reduction w/ Kernel Connection Multiplexor

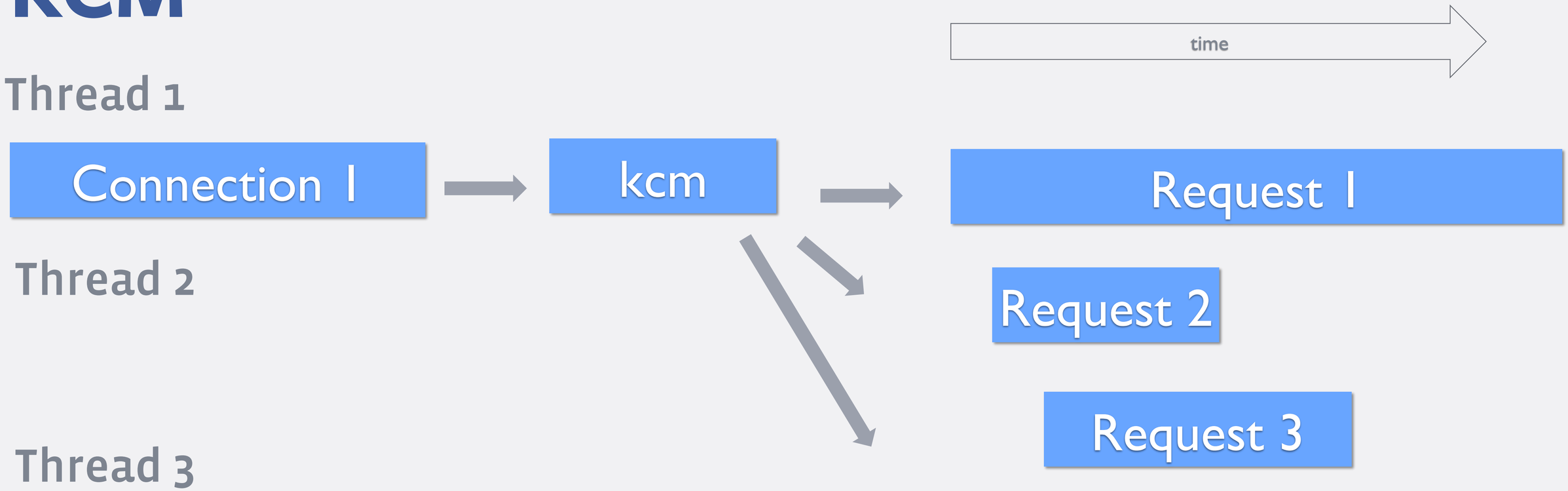


KCM

- Teach kernel about message boundaries using BPF
- Only wake on receipt of a full message
- Only wake a single thread that’s waiting

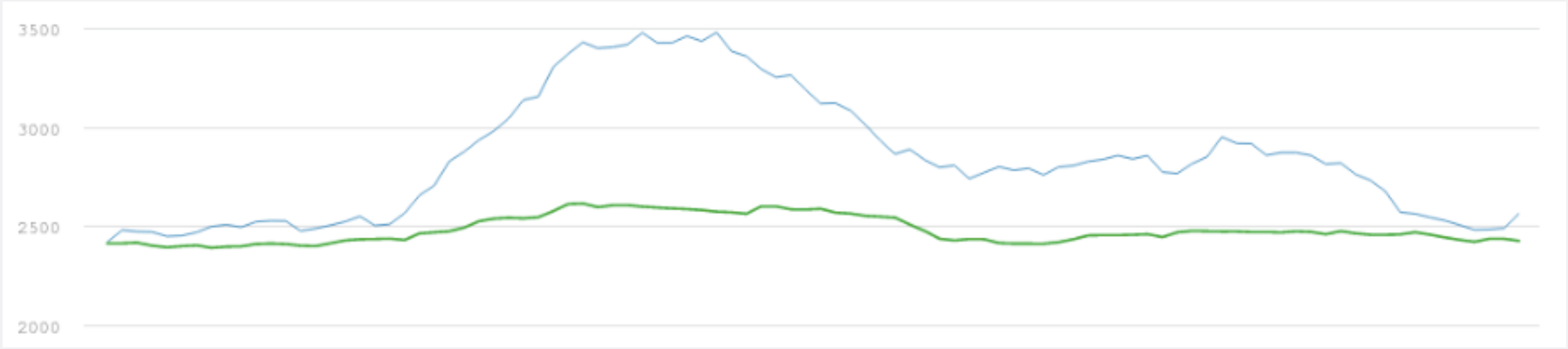
- Parallelize user space work for multiplexed data

KCM


time
Thread 1
Connection 1
kcm
Request 1
Thread 2
Request 2
Thread 3
Request 3
...

KCM + KTLS

Tail latency reduction



latency in ms

time ->

Why kernel TLS?

- Sendfile, splice, standard posix APIs
- Access to unencrypted bytes in kernel – e.g., BPF
- **Hardware Offload**

Offload

- AF_ALG– crypto interface – extra roundtrip to offload hardware
- socket – w/NIC offload, no extra latency

Why kernel TLS?

- Sendfile, splice, standard posix APIs
- Access to unencrypted bytes in kernel – e.g., BPF
- Hardware Offload

Workloads

Numbers from prod

- TLS terminators
  - ~10% CPU spent in AESNI
  - 1~2% spent in avoidable copyto/copyfrom user for CDN

*CDN is static content sent straight from disk to network

Handshakes

Asymmetric vs. symmetric crypto

- Handshakes –previous offload work
  - Greater than 50% resume rate
  - ~5% CPU

- Pooled connections in network
- Generally, asymmetric crypto more expensive

Implementation

Implementation

- Split control and data messages
- Parse framing
- Seq number handling
- Encryption / decryption
- Buffer management

Implementation

Split control messages and data

- 2 fds – TCP socket, AF_TLS socket
- 1 fd + msg_errqueue – only AF_TLS socket
  - 1 fd + stateful sockopt

Implementation

Parse framing

- net/strparser lib
- also used by KCM
- mismatch between streaming socket data and datagrams

Implementation

Sequence number management

- Sockopts back and forth to userspace
- Most of the userspace implementation

Implementation

Crypto API

- Software GCM, aesni GCM, offloads
- Would also need changes for AF_ALG

Implementation

Buffer management

- 4 pages buffer size to always decrypt

TL;DR

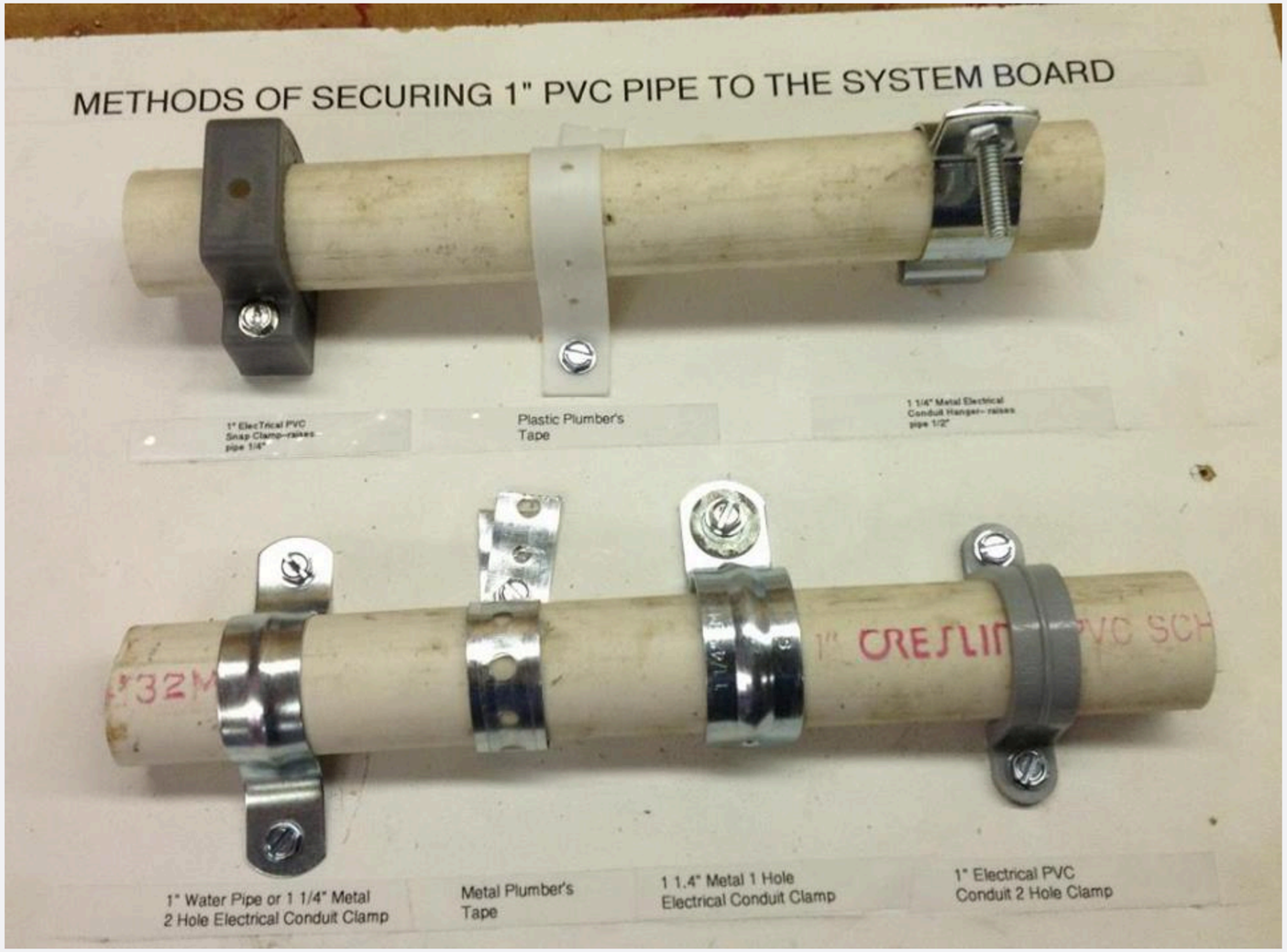
Conclusion

- 4-7% CPU savings in sendfile
- ~10-20% P99 latency drop w/KCM. Access to bytes in kernel
- Supports TLS 1.3, DTLS
- Provides path for NIC hardware offload

Status

- GCM/AES only
- DTLS work in progress

github.com/ktls/af_ktls



facebook